

CHEAT SHEET

Bidaw 1-페이지 Cheat Sheet

발표 5분 전 훑는 압축 요약

Bidaw — FAST '26 paper seminar 보조자료

Bidaw: Enhancing Key-Value Caching for Interactive LLM Serving via Bidirectional Computation-
Storage Awareness

Hu et al., FAST '26

발표 5분 전에 한 번 훑는 용도. **외워서** 들어가야 하는 것만 정리.

한 줄 요약

Compute engine과 two-tier storage(host DRAM + SSD)를 **양방향**으로 인식시켜 KV 로딩 병목을 해소. Compute → Storage : **previous answer length** · Storage → Compute : **KV location & size**

핵심 숫자 (틀리면 안 되는 것)

항목	값
평균 라운드 / 사용자 (자체 trace)	22.4
동시 캐시 KV (OPT-13B, 30 users/min)	480 GB (perf layer 200GB)
80% access의 reuse distance	> 200 GB (= perf layer)
기존 시스템 hit rate	≈ 20 % (FIFO/LRU/queue-enh)
답변길이 ↔ reuse distance 하한 ρ	0.94 – 0.98 (Spearman)
Tensor 6 cost efficiency	51 GFLOPs/MB (KV 30.5)
응답 latency 단축	최대 3.58 ×
Throughput 향상	1.43 – 1.83 ×
Tail (P90 / P95 / P99)	-53 / -49 / -47 %
Eviction miss rate	-57.6 % vs queue-enh · -69.9 % vs FIFO/LRU
Scheduling overhead	0.62 ms · Eviction overhead 0.35 ms

외워야 할 두 식

```
disk-HRRN      R = 1 + waiting_time / KV_size
                                     (preparing-queue 우선순위)

Equation 2      Overall_potential
                = prob_small · 1.0
                + prob_extreme · 0.0
                + Σ_i prob_promising(i) · hit_promising(i)
                                     (가장 낮은 KV를 evict)
```

세 가지 관찰 — 한 줄씩

1. 동시 캐시 KV가 perf layer 초과 — 22.4 라운드 × 사용자 수 × KV 크기 → SSD까지 사용 필연.
2. 약한 temporal locality — 80%가 perf layer 크기 초과, 어떤 정책도 hit ≈ 20 %.
3. Loading time 분산이 큼 — CV > 90 % (5초 윈도우), KV size + bandwidth gap의 합작.

세 가지 메커니즘 — 한 줄씩

#	이름	한 줄
1	I/O-aware scheduling	Dual queue (ready / preparing) + disk-HRRN로 큐잉 시간 5.76→2.45s
2	Previous-answer-based eviction	답변 길이 → reuse distance 하한 → bucket 확률 truncate → ghost-cache + Belady로 hit potential 계산
3	Storage-efficient tensor caching	KV 대신 Tensor 6 (정규화된 activation) 캐시, 변환은 low-priority CUDA stream — MHA 한정

Ablation

vanilla → +scheduling 1.58× → +eviction 1.97× → +tensor 2.17×

자주 헷갈리는 디테일

- **Weighted reuse distance** = "두 access 사이에 끼어든 다른 KV의 **byte 합**". 횡수 거리가 아님.
- **Promotion** 시 ready queue 위치 = **원래 도착 시각** 기준. promotion 시각 아님.
- Belady가 ghost cache에서 가능한 이유 — 이미 **지나간** trace이므로 사후 시뮬.

- **답변 길이 ↔ reuse distance**는 **하한** 단조 관계, 평균/상한 아님.
- **Inclusive caching** = capacity layer에 항상 사본 → eviction = pointer 제거 (write 0).
- **GQA에서는 Mechanism 3 비활성**. KV 자체가 작아 변환 비용이 우위 잡아먹음.

비교 baseline

vLLM	= recompute (no caching)
CachedAttention	= queue-enhanced LRU + 2-tier (ATC '24)
FlashGen	= inclusive caching + GPU-fit scheduling (FAST '25)
HCache	= intermediate activation 캐싱 (EuroSys '25)
Optimal	= perf-layer-only 가정 (ideal upper bound)

한계 4가지 (반드시 마지막에 짚기)

1. Single-user KV reuse — cross-user는 범위 밖 (MeanCache와 직교).
2. GQA에서 Mechanism 3 비활성.
3. ShareGPT처럼 timestamp 부재 trace에선 eviction 효과 약화.
4. Single GPU 노드 평가 — disaggregated/multi-node 미평가.

실험 환경 (Q&A 대비)

GPU	: NVIDIA A800 80 GB (1대)
Host mem	: 200 GB DRAM (perf layer)
Storage	: RAID-5 over 4× SATA SSD, 1.5 GB/s (capacity)
PCIe	: Gen 4 (~30 GB/s)
Models	: OPT-6.7B/13B/30B, Qwen-7B/14B
Workload	: 자체 1M+ round trace · ShareGPT (Poisson)

30초 elevator pitch

"Bidaw는 인터랙티브 LLM 서빙에서 host memory + SSD 2-tier KV cache의 로딩 병목을 푸는 시스템입니다. 핵심 통찰은 compute와 storage가 서로의 정보를 안 본다는 거고, 두 방향으로 정보를 흘립니다. Storage → compute로는 KV의 위치와 크기를, compute → storage로는 직전 답변의 길이를 보냅니다. 답변 길이는 사용자 사고 시간을 예측하므로 KV의 다음 access reuse distance 하한을 추정하는 신호가 됩니다. 다섯 모델에서 평균 latency 3.58× 단축, throughput 1.83× 향상으로 ideal 캐싱 상한에 거의 붙습니다."